

Barriers

Group synchronization at phase boundaries

Definition

A **barrier** is a synchronization mechanism that blocks each process at the end of a phase until all processes in the group have reached the same point.

1. Why Barriers Are Used

Main Idea

Some parallel applications are divided into phases. No process may begin the next phase until every process has finished the current phase.

Barriers are different from producer-consumer synchronization. Producer-consumer usually coordinates two sides exchanging data. A barrier coordinates a whole group of processes that must move through a computation together.

Phase Rule

All processes must complete phase n before any process starts phase $n + 1$.

2. Basic Barrier Operation

Barrier Primitive

A process usually reaches a barrier by calling a library procedure such as:

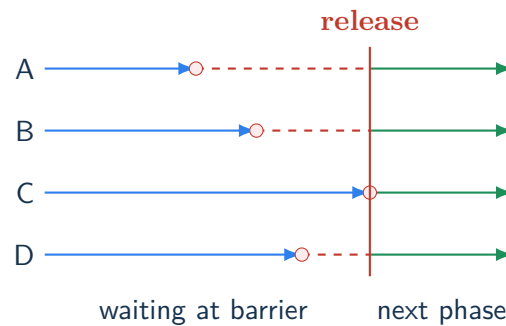
```
barrier()
```

Event	Result
Process reaches barrier early	It is blocked and waits.
Other processes arrive	They also block at the barrier.
Last process arrives	All blocked processes are released together.

Blocking Point

A process that reaches the barrier cannot continue into the next phase by itself. It must wait for the entire group.

3. Barrier Timeline



Reading the Diagram

Processes A, B, and D reach the barrier early and wait. Process C is last. When C arrives, the barrier opens and all processes move into the next phase.

4. Step-by-Step Behavior

Stage	Description
(a) Approaching	Processes are still computing during the current phase.
(b) Waiting	Some processes have reached the barrier and are blocked.
(c) Release	The final process reaches the barrier, so all processes are allowed through.

Group Synchronization

A barrier is useful when the correctness of the next phase depends on every process having completed its current work.

5. Example: Matrix Relaxation

Relaxation Problem

A **relaxation problem** repeatedly updates a large matrix to simulate how a physical system changes over time.

Temperature Simulation

Imagine a matrix whose entries represent temperatures at points on a metal sheet. A flame is placed at one corner, and the program computes how heat spreads through the sheet over time.

Starting from the current matrix, the program applies a transformation to produce the next matrix. For example, it may compute the temperatures after a small time step ΔT . Repeating this produces a sequence of matrices over time.

6. Why the Relaxation Problem Needs Barriers

Parallel Work

If the matrix is extremely large, different processes can compute different parts of the next matrix in parallel.

Main Danger

No process should start iteration $n + 1$ before all processes have finished iteration n . Otherwise, some processes may use old values while others are already using new values.

Barrier Placement

Each process executes `barrier()` after finishing its part of the current iteration. When all processes arrive, the new matrix is complete and everyone can start the next iteration.

7. Matrix Relaxation Sketch

```
for iteration = 0 to max_iterations - 1 do
  compute assigned part of next_matrix from current_matrix

  barrier()

  make next_matrix the current_matrix for the next iteration

  barrier()
end for
```

Why Two Barriers May Appear

The first barrier ensures that all pieces of `next_matrix` are finished. A second barrier may be needed if processes must all finish switching to the new matrix before any process computes again.

8. Barrier Guarantees

Guarantee	Meaning
No early entry	No process enters the next phase before all processes reach the barrier.
Group release	When the last process arrives, all waiting processes can continue.
Phase ordering	The computation proceeds in clear rounds or iterations.

9. What Barriers Do Not Do

Not Mutual Exclusion

A barrier does not protect a critical section. It does not say that only one process may access shared data at a time.

Not Message Passing

A barrier does not itself transfer data between processes. It only coordinates when processes may continue.

Different Purpose

Mutexes, semaphores, and monitors usually protect shared objects or coordinate specific events. Barriers synchronize an entire group at a phase boundary.

10. Simple Implementation Idea

Counter-Based Barrier

A basic barrier can keep a count of how many processes have arrived. If the count is less than the group size, the process waits. If the count reaches the group size, all waiting processes are released.

```
barrier()
  arrived := arrived + 1

  if arrived < number_of_processes then
    wait until barrier opens
  else
    open barrier and release all waiting processes
  end if
end barrier
```

Reusable Barrier

Real barriers are often reused in every iteration, so the implementation must reset safely after all processes have passed through.

11. Common Barrier Problem

Missing Process

If one process crashes, runs forever, or never calls `barrier()`, all other processes may wait at the barrier forever.

Load Imbalance

If one process has much more work than the others, the faster processes spend more time waiting at the barrier. This can reduce parallel performance.

12. Barriers vs. Other Synchronization

Mechanism	Main Use	Typical Scope
Mutex	Protect one critical section.	One shared object or code region.
Semaphore	Count resources or coordinate events.	One resource set or event pattern.
Monitor	Protect shared data with automatic mutual exclusion.	One protected module.
Message passing	Send data and synchronize without shared memory.	Processes or mailboxes.
Barrier	Wait until a whole group reaches the same phase boundary.	A group of cooperating processes.

13. Big Picture

Main Conclusion

Barriers provide group synchronization. When processes are divided into phases, a barrier ensures that no process begins the next phase until all processes have completed the current one.

Exam Shortcut

Remember: a barrier blocks early arrivals, waits for the last process, and then releases the whole group together.

14. Quick Review

Key Points

- Barriers are intended for groups of processes.
- They are useful when a program is divided into phases.
- A process calls `barrier()` at the end of a phase.
- Early processes block at the barrier.
- When the last process arrives, all processes are released.
- Barriers prevent any process from starting phase $n + 1$ too early.
- Matrix relaxation problems often need barriers between iterations.
- A barrier does not provide mutual exclusion by itself.
- If one process never reaches the barrier, the others may wait forever.
- Load imbalance can make fast processes spend time waiting at barriers.